RaspFormer: What the Experiments Show

Main finding. PR and CKA can detect some stages where a program variable is first written. High timing scores can still occur when the model gives wrong answers. Complete algorithm recovery remains unproven.

1 Question and study setup

The question is whether representation measurements reveal a model's computation. The original benchmark checks independently evaluated RASP values using compiler-known coordinates and write times. That is a useful fidelity check, but agreement alone does not discover the mechanism.

The follow-up studies add coordinate changes, executing compression, blind write detection, causal patches, and models trained on the same tasks. The twelve tasks include arithmetic, counting, shifts, order checking, palindrome checking, and sorting.

Study	Data and controls
Compiled tasks	Twelve RASP programs; vocabulary −2 to 5; length six; noncausal attention. BOS is excluded from token scores.
Matched fidelity	3,383 unique sequences at six positions: 20,298 aligned observations per task.
Compiled controls	96 calibration and 96 disjoint test sequences. Fitted bases and patch-stage choices use calibration data.
Structural controls	Nine variants cross requested branch depth and branch count.
Learned models	Twelve tasks, three seeds each: 36 models. Whole-sequence train/validation/test split: 2,029/677/677.

What the measurements mean

Participation ratio (PR) describes how many directions carry variation. It depends on the encoding, data distribution, and feature scale. A balanced, equally scaled one-hot variable with k categories has PR $k-1$. This explains values 5, 1, and 12 for the original one-variable examples P01, P02, and P03. PR is not an independent measure of algorithmic complexity.

Linear centered kernel alignment (CKA) measures similarity between two aligned activation matrices [2]. Similarity can be useful evidence, but it does not identify the underlying algorithm.

Principal component analysis (PCA) picks directions with the most variation. A patch replaces part of one activation with activity from another input to test its effect.

First-write detection asks when at least one variable first appears. It does not recover variable names, how many variables are written together, or how long they remain useful.

All findings below use saved reference experiments. Comparisons are descriptive; averages give each program equal weight unless stated otherwise. No scientific experiment was rerun to prepare this shorter report.

2 Write timing and correct answers

Blind detection finds a useful timing signal

Detectors receive ordered activation matrices without compiler schedules, lane maps, or variable names. Predictions are saved and hashed before schedule-based scoring. P07 to P12 are the six held-out programs. Thresholds are fixed; they are not fitted on the designated P01 to P06 calibration program group.

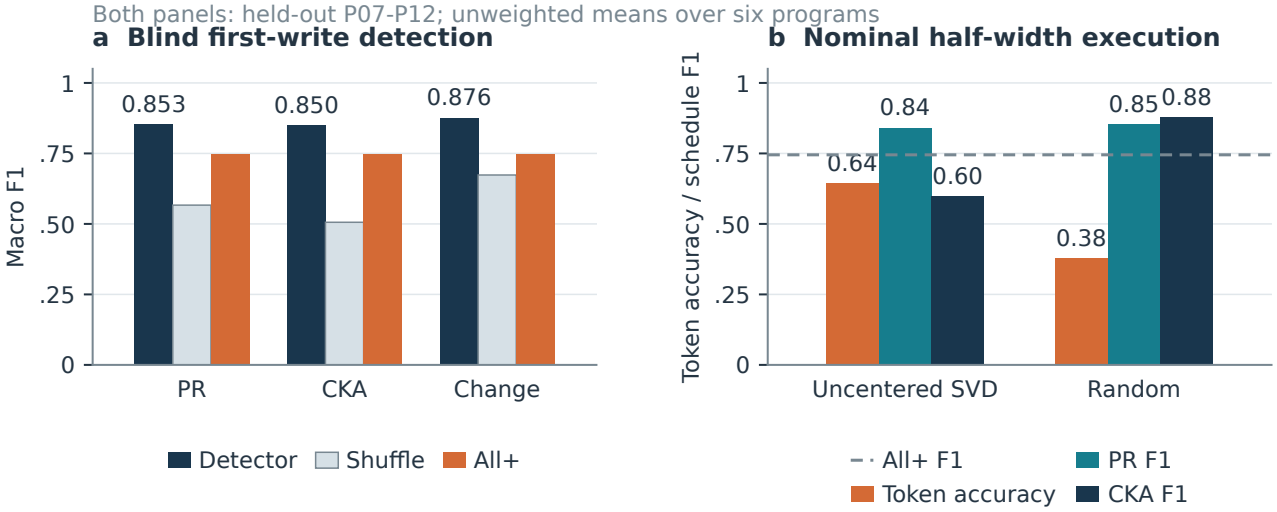


Figure 1. Both panels show unweighted means over the same six held-out programs, P07 to P12. (a) First-write F1 compared with count-matched shuffles and always predicting a write (All+). (b) Nominal half-width models execute with compressed residual states. High agreement with the original intended schedule does not establish preserved computation.

PR and CKA achieve macro F1 of **0.853** and **0.850**, above the always-write baseline of 0.745. A simpler residual-change detector scores **0.876**. The shuffled comparison uses 1,000 within-program permutations that preserve the number of predicted events. P11 CKA scores 0.727, below its own 0.737 baseline.

A high timing score does not guarantee correct answers

On the held-out six, random half-width compression gives **0.879 CKA schedule F1** while mean task accuracy is only **37.62%**. These are group means over six programs. Once behavior changes, the original schedule is an intended reference, not known causal ground truth for the compressed model.

Across *all twelve* tasks, half-width uncentered singular value decomposition (SVD) gives 61.62% token accuracy, random projection 37.01%, and full rank 100%. This population differs from the six-program figure. The saved compression label `pca` means uncentered SVD; intervention PCA is centered. Residual rank is $\lceil df \rceil$ for width d and target fraction f. Attention and MLP hidden widths remain fixed.

Meaning. Geometry detects some changes in execution timing. The compression result shows why that evidence cannot, by itself, establish correct computation or recovered semantic superposition.

3 Numerical errors and causal patches

The variance floor removes misleading scores

A representation is active only when total centered sample variance exceeds 10^{-12} (ddof 1). Raw variance and spectra remain saved. Inactive PR is zero by convention; CKA is undefined.

This reduces CKA availability mismatches from **14 to 2** across 54 matched computed-view transitions. Twelve early stages have variance near 8.07×10^{-19} despite raw PR near one. The remaining two mismatches involve helper-state timing. The floor is an absolute, scale-dependent numerical policy.

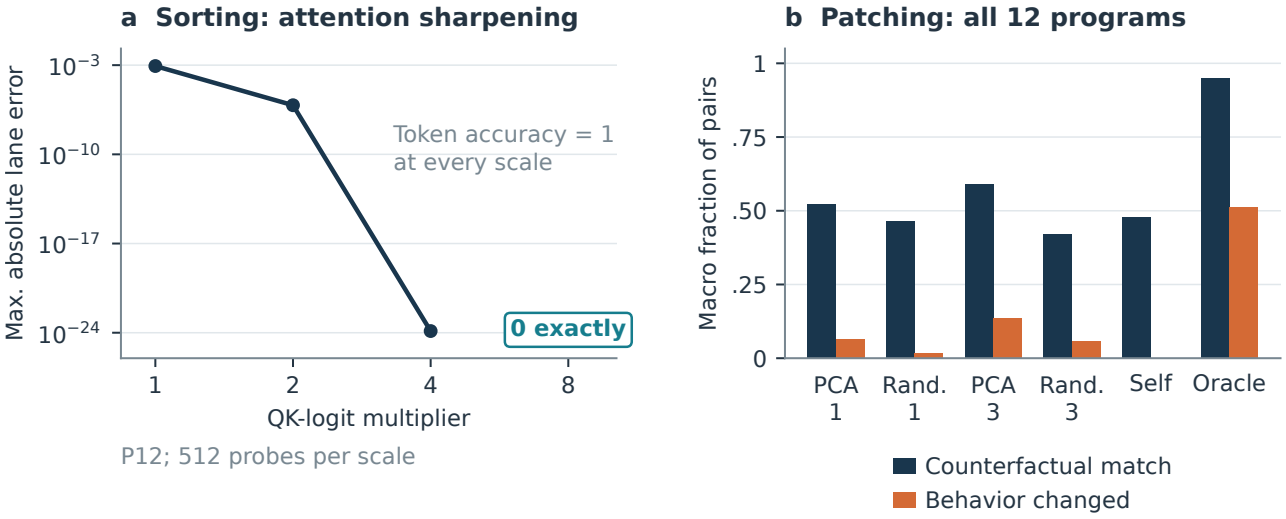


Figure 2. (a) Sharper attention reduces sorting's lane error on 512 probes; decoded accuracy stays 100%. The recorded zero at scale eight is shown separately from the log axis. (b) Patch results across all twelve programs use the same 192 held-out pairs in each condition. Oracle rank varies by variable. These are descriptive comparisons, not independent samples.

Sorting's equal error rates come from different rows

Each of eight output lanes fails 64 of 512 probes, giving 87.5% agreement. The failure sets are disjoint. Doubling QK-logit scale reduces maximum lane error from 8.52×10^{-4} to 7.15×10^{-7}, with 100% decoded accuracy throughout. This supports finite-attention approximation; ties or duplicates are not established as the sole cause.

PCA patches add limited causal evidence

Rank-three PCA patches produce the answer expected after changing one program variable on **113/192 pairs (58.85%)**, versus 42.19% for equal-rank random directions and 47.92% for self patches. They alter outputs in 26 pairs. Stage choice and PCA use calibration activations; mapping directions to named variables uses supervised labels.

Six conditions share 192 pairs, producing 1,152 correlated rows. Ten undefined P12 substitutions contribute 60 failed condition rows; none are discarded. The result supports a useful geometric intervention, while unsupervised variable recovery remains unproven.

4 Rotation, structural controls, and learned models

Rotation separates whole-space measurements from lane checks

All **36 executed orthogonal transforms** preserve 100% token and sequence agreement. Maximum posthoc PR and CKA errors are 1.42×10^{-13} and 2.22×10^{-15}. Fixed original-lane checks change after rotation.

This is the expected invariance of the metrics. It does not identify a computation. Finite-precision rotated execution has a separate maximum residual discrepancy of 3.65×10^{-5} while decoded outputs stay unchanged. Nonorthogonal rescaling changes PR, confirming its dependence on relative feature scale.

The structural design breaks exact collinearity

In the original twelve programs, variable count equals dependency depth. Nine new variants cross requested branch depth and branch count from one to three. The centered realized-depth/variable-count matrix has **rank two**, compared with rank one originally.

The variants span one to eleven computed variables, realized depth one to five, and residual width 24 to 105. All sampled outputs agree with RASP. Serial merges add depth; branch count controls active branching, and residual width changes as a consequence. This removes exact collinearity, but does not establish a general complexity law.

Learned models solve the tasks with different geometry

Thirty-six randomly initialized transformers train for 600 Adam steps, using width 32, two layers, two heads, batch size 128, and learning rate 0.003. All parameters train; each task uses three seeds.

Learned-model result	Observed outcome
Per-task mean token accuracy	95.41% to 100%; six tasks average 100%.
Palindrome	95.41% accuracy, versus a 93.94% majority baseline.
Increasing-check	98.94% accuracy, versus a 92.76% majority baseline.
Final learned/compiled CKA	0.152 to 0.802 across tasks.
Final participation ratio	Lower in learned models for every task.

High accuracy on imbalanced predicates needs a majority-class baseline. Learned and compiled stages are paired by relative progress, so their similarity does not establish matching variables. Saved checkpoints replay on all 677 test sequences.

Meaning. Models can produce similar answers using different representations. The trained models' internal causal circuits remain unknown. New sequence lengths and data distributions were not evaluated.

5 Conclusions, limits, and reproducibility

What the evidence supports

The follow-ups resolve several specific problems in the original evaluation: near-zero numerical artifacts, the sorting row-error hypothesis, exact depth/count collinearity, and dependence on fixed coordinate lanes. Blind timing tests and PCA patches add evidence beyond compiler-fidelity checks.

Supported by the saved experiments	Still unestablished
Partial detection of first-write events.	Complete algorithms, variable names, or causal circuits from PR/CKA alone.
Expected invariance under orthogonal rotation.	Reliable computation recovery after compression changes behavior.
Higher PCA patch agreement than the recorded random and self controls.	Unsupervised mapping of geometric directions to program variables.
Noncollinear structural examples and learned-model comparisons.	A general complexity law, erasure, overwriting, variable lifetimes, or broad generalization.

The original fidelity benchmark still uses compiler-known encodings and write times. PR error remains a secondary diagnostic. Correlation alone is also insufficient: sorting lanes have Pearson $r > 0.999999999996$ while only 87.5% of scalar values meet tolerance. Full and computed residual views remain useful, but computed views still include helper lanes.

Position relative to earlier work

Tracr provides models with known structure and already studies superposition [1]. InterpBench uses Strict Interchange Intervention Training (SIIT) to provide known causal circuits for evaluating discovery methods [3]. Our ordinary output-supervised models do not inherit that internal ground truth. No direct InterpBench benchmark was run.

Reproducible evidence

Scientific runs were recorded on 15 October 2026 and pinned to commit `e06479c` [4]. Saved verification includes 28 passing tests, five notebooks with 37 executed code cells, and replay of all 36 checkpoints. Manifests preserve dependencies, source hashes, Git state, numerical settings, and failures. This shorter PDF uses those saved results; it does not report new experiments.

Conclusion. PR and CKA reveal some execution timing. Numerical controls and causal tests make that finding more credible, while compression exposes its limits. Stronger algorithm-recovery claims need independent internal ground truth and broader causal validation.

References

[1] D. Lindner et al. *Tracr: Compiled Transformers as a Laboratory for Interpretability.* NeurIPS 2023. arXiv:2301.05062.

[2] S. Kornblith et al. *Similarity of Neural Network Representations Revisited.* ICML 2019. arXiv:1905.00414.

[3] R. Gupta et al. *InterpBench: Semi-Synthetic Transformers for Evaluating Mechanistic Interpretability Techniques.* NeurIPS 2024. doi:10.52202/079017-2950.

[4] RaspFormer project. *Canonical implementation and research evidence.* Commit `e06479c`. Code, data, predictions, and checkpoints.